

Multivariate Probability Distributions in R

Md. Mostakim

Student of Applied Statistics and Data Science
Institute of Applied Statistics and Data Science
University of Dhaka



Section 1

Multivariate Probability Distributions in R

What this presentation covers

This presentation develops a practical workflow for working with multivariate probability distributions in R:

Stage	Main idea	R tools
1	Recall univariate foundations	<code>dnorm()</code> , <code>pnorm()</code> , <code>rnorm()</code>
2	Understand joint behavior	joint, marginal, conditional distributions
3	Measure dependence	covariance, correlation matrices
4	Work with the multivariate normal	<code>mvrnorm()</code> , <code>dmvnorm()</code> , <code>rmvnorm()</code>
5	Handle other key distributions	Multinomial, Dirichlet, Wishart
6	Estimate parameters	MLE, method of moments

What this presentation covers (cont.)

7	Simulate and sample	<code>rmultinom()</code> , Cholesky decomposition
8	Visualize multivariate data	contour plots, <code>pairs()</code> , heatmaps
9	Apply to real data	dimension reduction, classification, portfolio modeling

Learning goal: *By the end, you should be able to describe the joint behavior of multiple random variables, quantify their dependence through covariance and correlation, generate samples from common multivariate distributions in R, and interpret their structure visually and numerically.*

A simple mental model

**Describe → Measure dependence → Simulate → Estimate →
Visualize → Apply**

This sequence is more important than memorizing individual R commands.

Joint distribution: describes the probability of two or more variables occurring together.

Marginal distribution: the distribution of one variable ignoring the others.

Conditional distribution: the distribution of one variable given fixed values of others.

Covariance matrix: summarizes the pairwise linear dependence between all variables.

Multivariate Normal: the cornerstone distribution defined by a mean vector and covariance matrix.

Multinomial: generalizes the Binomial to more than two outcome categories.

Dirichlet: a distribution over probability vectors, often used as a prior in Bayesian models.

Cholesky decomposition: a key trick for generating correlated random samples in R.

Section 2

Reading Multivariate Data

Reading multivariate data using read.table

Reading multivariate data from a file or an external URL can generally be done using the `read.table()` or `read.csv()` commands, depending on the type of data source. In this exercise you will read the wine dataset from the UCI machine learning website

```
# Read in the wine dataset
wine <- read.table(
  ↪ "https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data"
  ↪ ,
  sep = ",")

# Print the first four entries
head(wine, n = 4)
```

##	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14
## 1	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
## 2	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
## 3	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
## 4	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480

Reading multivariate data using read.table (cont.)

```
# Find the dimensions of the data  
dim(wine)
```

```
## [1] 178 14
```

```
# Check the names of the wine dataset  
names(wine)
```

```
## [1] "V1" "V2" "V3" "V4" "V5" "V6" "V7" "V8" "V9" "V10" "V11"  
↪ "V12"  
## [13] "V13" "V14"
```

Specifying datatypes for columns

When you read data from a text or CSV file, you should specify the names and data types for each column. The `read()` function will try to determine if the first entry of the dataset contains the column names. R is clever at figuring out some datatypes, but if you are reading a categorical variable coded as 0, 1, and 2, it will read it as a numeric variable, and you will need to specify the data type for that column after reading the data.

```
# Assign new names
colnames(wine) <- c("Type", "Alcohol", "Malic", "Ash", "Alcalinity",
  "Magnesium", "Phenols", "Flavanoids", "Nonflavanoids",
  "Proanthocyanins", "Color", "Hue", "Dilution", "Proline")
```

```
# Check the new column names
names(wine)
```

```
## [1] "Type"           "Alcohol"         "Malic"           "Ash"
## [5] "Alcalinity"     "Magnesium"       "Phenols"         "Flavanoids"
## [9] "Nonflavanoids" "Proanthocyanins" "Color"           "Hue"
## [13] "Dilution"      "Proline"
```

Specifying datatypes for columns (cont.)

```
# Check data type/structure of each variable
```

```
str(wine)
```

```
## 'data.frame':      178 obs. of  14 variables:
## $ Type           : int  1 1 1 1 1 1 1 1 1 1 ...
## $ Alcohol        : num  14.2 13.2 13.2 14.4 13.2 ...
## $ Malic          : num  1.71 1.78 2.36 1.95 2.59 1.76 1.87 2.15 1.64
  ↪ 1.35 ...
## $ Ash            : num  2.43 2.14 2.67 2.5 2.87 2.45 2.45 2.61 2.17 2.27
  ↪ ...
## $ Alcalinity     : num  15.6 11.2 18.6 16.8 21 15.2 14.6 17.6 14 16 ...
## $ Magnesium      : int  127 100 101 113 118 112 96 121 97 98 ...
## $ Phenols        : num  2.8 2.65 2.8 3.85 2.8 3.27 2.5 2.6 2.8 2.98 ...
## $ Flavanoids     : num  3.06 2.76 3.24 3.49 2.69 3.39 2.52 2.51 2.98
  ↪ 3.15 ...
## $ Nonflavanoids  : num  0.28 0.26 0.3 0.24 0.39 0.34 0.3 0.31 0.29 0.22
  ↪ ...
## $ Proanthocyanins: num  2.29 1.28 2.81 2.18 1.82 1.97 1.98 1.25 1.98
  ↪ 1.85 ...
## $ Color          : num  5.64 4.38 5.68 7.8 4.32 6.75 5.25 5.05 5.2 7.22
  ↪ ...
## $ Hue            : num  1.04 1.05 1.03 0.86 1.04 1.05 1.02 1.06 1.08
  ↪ 1.01 ...
```

Specifying datatypes for columns (cont.)

```
## $ Dilution      : num  3.92 3.4 3.17 3.45 2.93 2.85 3.58 3.58 2.85 3.55
↪ ...
## $ Proline       : int   1065 1050 1185 1480 735 1450 1290 1295 1045 1045
↪ ...
```

Change the Type variable data type

```
wine$Type <- as.factor(wine$Type)
```

Check data type/structure again

```
str(wine)
```

```
## 'data.frame':    178 obs. of  14 variables:
## $ Type          : Factor w/ 3 levels "1","2","3": 1 1 1 1 1 1 1 1 1 1
↪ ...
## $ Alcohol       : num   14.2 13.2 13.2 14.4 13.2 ...
## $ Malic        : num    1.71 1.78 2.36 1.95 2.59 1.76 1.87 2.15 1.64
↪ 1.35 ...
## $ Ash          : num    2.43 2.14 2.67 2.5 2.87 2.45 2.45 2.61 2.17 2.27
↪ ...
## $ Alcalinity   : num    15.6 11.2 18.6 16.8 21 15.2 14.6 17.6 14 16 ...
## $ Magnesium    : int     127 100 101 113 118 112 96 121 97 98 ...
## $ Phenols      : num     2.8 2.65 2.8 3.85 2.8 3.27 2.5 2.6 2.8 2.98 ...
## $ Flavanoids   : num     3.06 2.76 3.24 3.49 2.69 3.39 2.52 2.51 2.98
↪ 3.15 ...
```

Specifying datatypes for columns (cont.)

```
## $ Nonflavanoids : num 0.28 0.26 0.3 0.24 0.39 0.34 0.3 0.31 0.29 0.22
↪ ...
## $ Proanthocyanins: num 2.29 1.28 2.81 2.18 1.82 1.97 1.98 1.25 1.98
↪ 1.85 ...
## $ Color          : num 5.64 4.38 5.68 7.8 4.32 6.75 5.25 5.05 5.2 7.22
↪ ...
## $ Hue            : num 1.04 1.05 1.03 0.86 1.04 1.05 1.02 1.06 1.08
↪ 1.01 ...
## $ Dilution      : num 3.92 3.4 3.17 3.45 2.93 2.85 3.58 3.58 2.85 3.55
↪ ...
## $ Proline        : int 1065 1050 1185 1480 735 1450 1290 1295 1045 1045
↪ ...
```

Section 3

Mean vector and Variance-covariance matrix

Calculating the mean vector

The first step in analyzing multivariate data is computing the mean vector. The wine dataset consists of several variables. You will calculate the mean vector of the first four numeric variables, Alcohol, Malic, Ash, Alcalinity, which are located in columns 2 through 5. When observations in a dataset have different subgroups, like wine type, it is also helpful to calculate the mean vector by group.

```
# Calculate the mean of the Alcohol, Malic, Ash, and  
# Alcalinity variables  
colMeans(wine[, c("Alcohol", "Malic", "Ash", "Alcalinity")])
```

```
##      Alcohol      Malic      Ash Alcalinity  
## 13.000618    2.336348    2.366517    19.494944
```

```
# Calculate the mean of the variables by wine type  
by(data = wine[, c("Alcohol", "Malic", "Ash", "Alcalinity")],  
    INDICES = wine$Type, FUN = colMeans)
```

Calculating the mean vector (cont.)

```
## wine$Type: 1
##      Alcohol      Malic      Ash Alkalinity
## 13.744746    2.010678    2.455593    17.037288
## -----
## wine$Type: 2
##      Alcohol      Malic      Ash Alkalinity
## 12.278732    1.932676    2.244789    20.238028
## -----
## wine$Type: 3
##      Alcohol      Malic      Ash Alkalinity
## 13.153750    3.333750    2.437083    21.416667
```

Calculating the variance-covariance matrix

Along with the mean, an equally important statistic for a multivariate observation is its variance-covariance matrix. The variance-covariance matrix generalizes the concept of variance to multiple dimensions.

```
# Calculate the variance-covariance matrix of the  
# variables Alcohol, Malic, Ash, Alcalinity  
var.wine <- var(wine[, c("Alcohol", "Malic", "Ash", "Alcalinity")])
```

```
# Round the matrix values to two decimal places  
round(var.wine, 2)
```

```
##           Alcohol Malic  Ash Alcalinity  
## Alcohol      0.66  0.09  0.05      -0.84  
## Malic        0.09  1.25  0.05       1.08  
## Ash          0.05  0.05  0.08       0.41  
## Alcalinity  -0.84  1.08  0.41      11.15
```

Calculating the correlation matrix

A correlation matrix is a generalization of the concept of correlation between two variables. If you are looking at a large number of variables, it can also be useful to visualize the correlation matrix to understand the dependence of the variables on each other.

```
# Calculate the correlation matrix
cor.wine <- cor(wine[, c("Alcohol", "Malic", "Ash", "Alcalinity")])

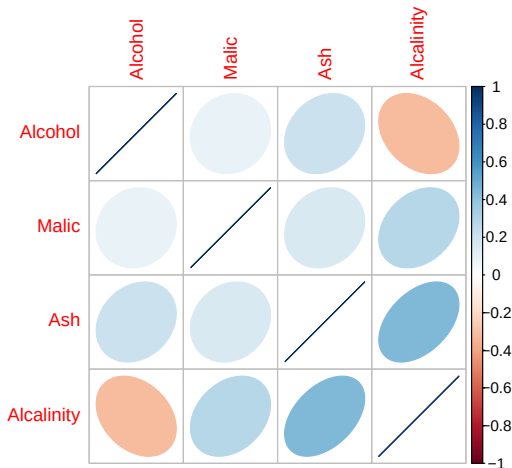
# Round the matrix to two decimal places
round(cor.wine, 2)
```

```
##           Alcohol Malic  Ash Alcalinity
## Alcohol         1.00  0.09  0.21      -0.31
## Malic           0.09  1.00  0.16        0.29
## Ash             0.21  0.16  1.00        0.44
## Alcalinity     -0.31  0.29  0.44        1.00
```

Calculating the correlation matrix (cont.)

```
# Plot the correlations  
  
library(corrplot)  
  
corrplot(cor.wine, method = "ellipse")
```

Calculating the correlation matrix (cont.)



How to read it

- Color indicates direction and strength:

Calculating the correlation matrix (cont.)

- Blue = positive correlation
- Red = negative correlation
- Darker color = stronger correlation

► Shape indicates strength

- Narrow ellipse = strong correlation
- Round circle = weak correlation

Section 4

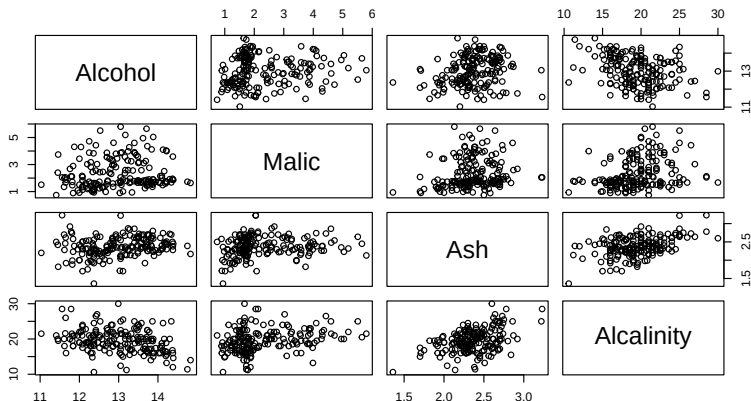
Plotting Multivariate Data

Pairs plot using base graphics and lattice

Visualizing multivariate data is very important to do before performing any multivariate analyses. In the following few exercises, you will explore a subset of these functions and evaluate the strengths and weaknesses of these plotting techniques.

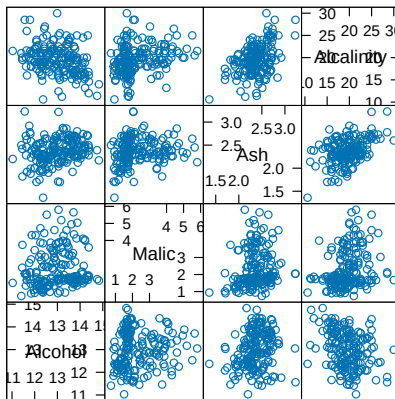
```
# Scatter plot matrix using the base R plot function  
pairs(wine[, 2:5])
```

Pairs plot using base graphics and lattice (cont.)



```
# Scatter plot matrix with lattice  
library(lattice)  
splom(~wine[, 2:5])
```

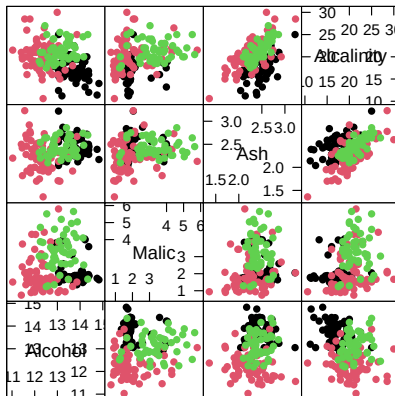
Pairs plot using base graphics and lattice (cont.)



Scatter Plot Matrix

```
# Scatter plot matrix colored by groups  
splom(~wine[, 2:5], pch = 16, col = wine$Type)
```

Pairs plot using base graphics and lattice (cont.)

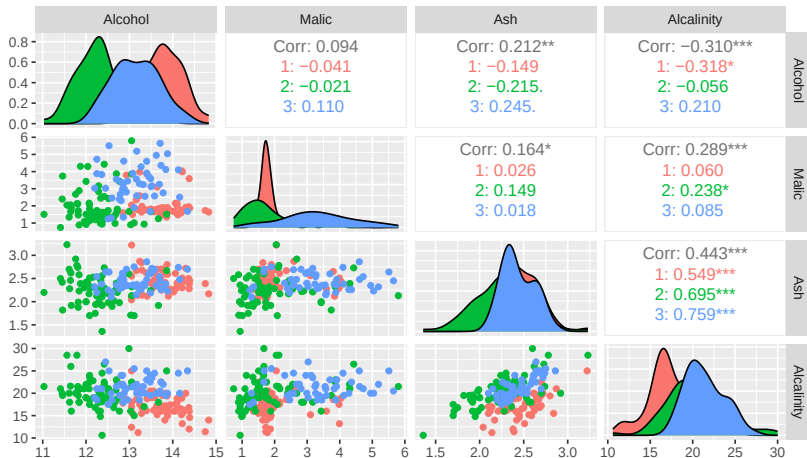


Scatter Plot Matrix

```
library(GGally)

ggpairs(data = wine, columns = 2:5, aes(colour = Type))
```

Pairs plot using base graphics and lattice (cont.)



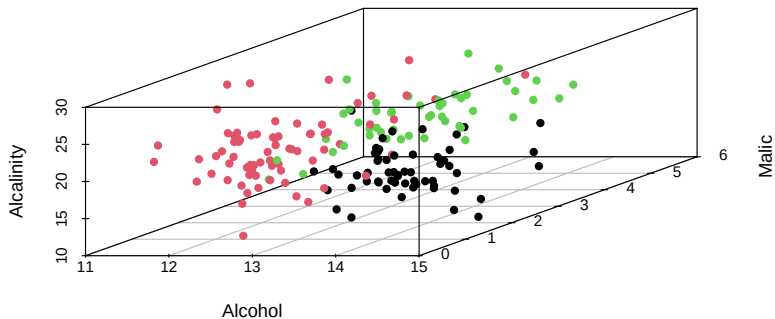
3D plotting techniques

Beyond the 2D plotting techniques, there are also several 3D plotting techniques that you can use to plot multivariate data. You will use the `scatterplot3d()` function from the `scatterplot3d` package to produce some basic 3D plots for the wine dataset.

```
library(scatterplot3d)

# Plot the three variables and color them using wine
# Type
scatterplot3d(wine[, c("Alcohol", "Malic", "Alcalinity")],
              color = wine$Type, pch = 16)
```

3D plotting techniques (cont.)

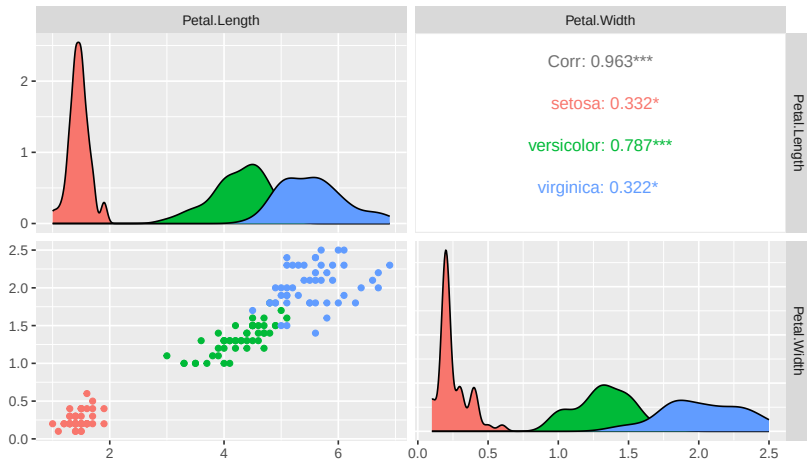


Section 5

Special for me

Summary of iris data using plot

```
ggpairs(iris, columns = 3:4, aes(colour = Species))
```



Summary of iris data using plot (cont.)

```
by(iris[, 1:4], INDICES = iris$Species, FUN = colMeans)
```

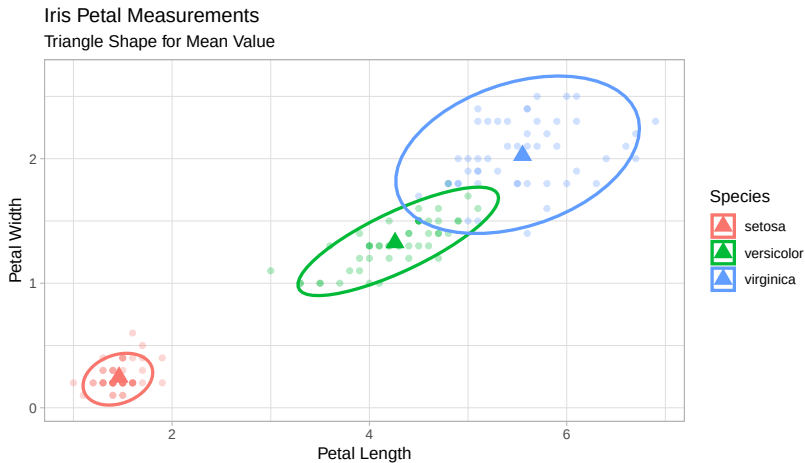
```
## iris$Species: setosa
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##          5.006          3.428          1.462          0.246
## -----
## iris$Species: versicolor
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##          5.936          2.770          4.260          1.326
## -----
## iris$Species: virginica
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##          6.588          2.974          5.552          2.026
```

Summary of iris data using plot (cont.)

```
data <- data.frame(Species = c("setosa", "versicolor", "virginica"),
  Petal_Length_mean = c(1.462, 4.26, 5.552), Petal_Width_mean = c(0.246,
    1.326, 2.026))

ggplot(iris, aes(x = Petal.Length, y = Petal.Width, color = Species,
  fill = Species)) + geom_point(alpha = 0.3) + stat_ellipse(level = 0.95,
  geom = "polygon", alpha = 0, lwd = 1) + geom_point(data = data,
  aes(x = Petal_Length_mean, y = Petal_Width_mean), size = 4,
  shape = 17) + labs(title = "Iris Petal Measurements",
  subtitle = "Triangle Shape for Mean Value", x = "Petal Length",
  y = "Petal Width") + theme_light()
```

Summary of iris data using plot (cont.)



Section 6

Multivariate Normal Distribution

Samples from multivariate normal distributions

```
mu.sim <- c(2, -2)
sigma.sim <- matrix(c(9, 5, 5, 4), nrow = 2)

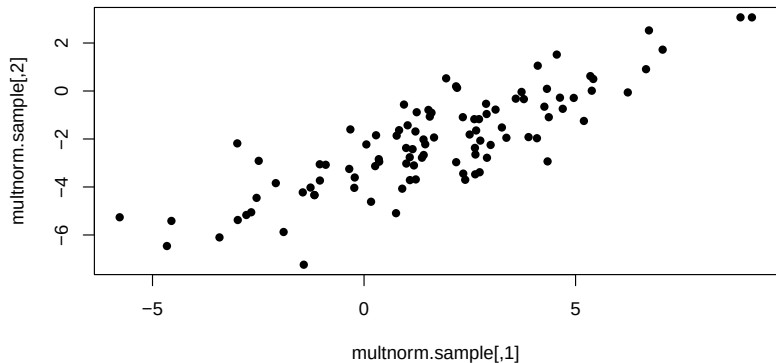
# Generate 100 bivariate normal samples
library(mvtnorm)
multnorm.sample <- rmvnorm(n = 100, mean = mu.sim, sigma = sigma.sim)

# View the first 6 samples
head(multnorm.sample)

##           [,1]      [,2]
## [1,]  1.940704  0.5275016
## [2,]  1.554278 -1.0679257
## [3,] -1.046427 -3.0541500
## [4,] -1.264800 -4.0249278
## [5,]  2.905284 -2.7830524
## [6,]  1.079896 -2.7615986

# Scatterplot of the bivariate samples
plot(multnorm.sample, pch = 16)
```

Samples from multivariate normal distributions (cont.)



Section 7

Density of Multivariate Normal Distribution

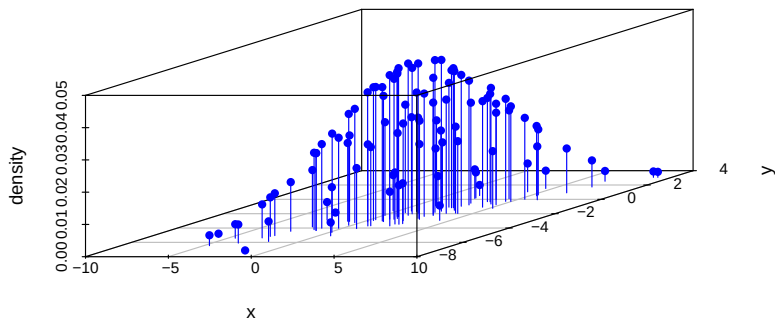
Calculating the density of multivariate normal

For many statistical tasks, like hypothesis testing, clustering, and likelihood calculation, you are required to calculate the density of a specified multivariate normal distribution. In this exercise, you will use the `dmvnorm()` function to calculate multivariate normal densities with specified mean and variance-covariance matrix at each of the observations from your previously generated sample `multnorm.sample`.

```
# Calculate density
multnorm.dens <- dmvnorm(multnorm.sample, mean = mu.sim,
  sigma = sigma.sim)

# Create scatter plot of density heights
scatterplot3d(cbind(multnorm.sample, multnorm.dens), color = "blue",
  pch = 16, type = "h", xlab = "x", ylab = "y", zlab = "density")
```

Calculating the density of multivariate normal (cont.)



Calculating dmvnorm over a grid

To visualize a bivariate normal density surface, one has to calculate the density over a dense grid of x and y coordinates, and use 3D surface plotting functions like `persp()` to visualize the surface.

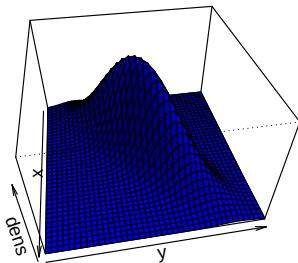
The 40 by 40 dense grid generated from the command

```
mvals <- expand.grid(seq(-5, 10, length.out = 40), seq(-8,
  4, length.out = 40))

# Calculate density over the specified grid
mvds <- dmvnorm(as.matrix(mvals), mean = mu.sim, sigma = sigma.sim)
matrix_mvds <- matrix(mvds, nrow = 40)

# Create a perspective plot
persp(matrix_mvds, theta = 80, phi = 40, expand = 0.6, shade = 0.2,
  col = "blue", xlab = "x", ylab = "y", zlab = "dens")
```

Calculating dmvmnorm over a grid (cont.)



Section 8

Cumulative Distribution and Inverse CDF

Using the pmvnorm function

Along with the density of multivariate normals, you often need to calculate the cumulative distributions of multivariate normals to obtain the volume of the density between two specified values. In this exercise, you will use the `pmvnorm()` function to calculate the cumulative distribution for specified bivariate normals.

```
# Volume under a bivariate standard normal  
pmvnorm(lower = c(-1, -1), upper = c(1, 1))
```

```
## [1] 0.4660649  
## attr("error")  
## [1] 1e-15  
## attr("msg")  
## [1] "Normal Completion"
```

```
# Volume under specified mean and variance-covariance  
# matrix  
pmvnorm(lower = c(-5, -5), upper = c(5, 5), mean = mu.sim,  
         sigma = sigma.sim)
```

Using the pmvnorm function (cont.)

```
## [1] 0.7734162
## attr("error")
## [1] 1e-15
## attr("msg")
## [1] "Normal Completion"
```

Calculating probability contours using qmvnorm

The inverse problem to the calculation of cumulative probability is as follows: for a given a probability calculate the contour that contains proportion of the total volume of the density. This contour is the same as the quantile of the distribution. The `qmvnorm()` function provides the tools to perform the above calculations.

```
# Probability contours for a standard bivariate normal  
qmvnorm(0.9, tail = "both", sigma = diag(2))
```

```
## $quantile  
## [1] 1.948779  
##  
## $f.quantile  
## [1] -1.537507e-06  
##  
## attr("message")  
## [1] "Normal Completion"
```

```
# Probability contours for a bivariate normal  
qmvnorm(0.95, tail = "both", mean = mu.sim, sigma = sigma.sim)
```

Calculating probability contours using qmvnorm (cont.)

```
## $quantile
## [1] 7.110635
##
## $f.quantile
## [1] 5.712626e-06
##
## attr("message")
## [1] "Normal Completion"
```

Section 9

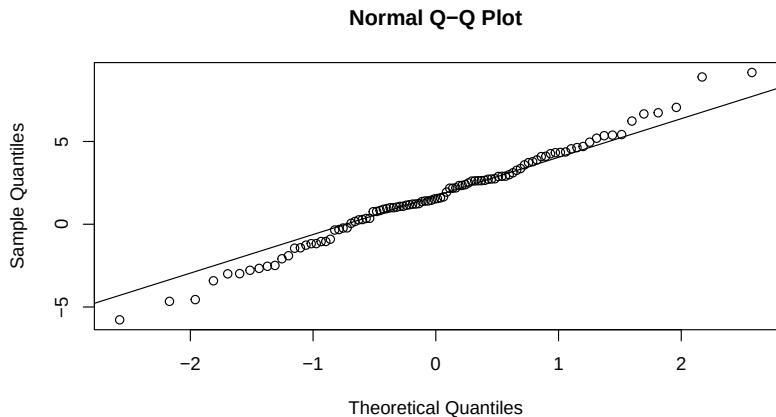
Checking normality of multivariate data

Graphical tests for multivariate normality

You are often required to verify that multivariate data follow a multivariate normal distribution. Recall that univariate normality of each individual variable does not imply multivariate normality overall. We will use functions to check multivariate normality of all variables instead of univariate normality of single variables.

```
# Test sample normality  
qqnorm(multnorm.sample[, 1])  
qqline(multnorm.sample[, 1])
```

Graphical tests for multivariate normality (cont.)



Graphical tests for multivariate normality (cont.)

```
library(MVN)

# Mardia's test qqplot
wine.mvntest <- mvn(wine[, 2:5], mvn_test = "mardia")
wine.mvntest$multivariate_normality
```

##		Test Statistic	p.value	Method	MVN
## 1	Mardia Skewness	80.606	<0.001	asymptotic	Not normal
## 2	Mardia Kurtosis	0.559	0.576	asymptotic	Normal

Section 10

Multivariate t-distribution

Generate samples from multivariate t-distribution

Though multivariate normal is widely used, not all multivariate data follow a normal distribution. Multivariate t-distributions can accommodate heavy-tailed distribution in each direction. In this exercise, you will learn how to draw random samples from a multivariate t-distribution. We will use the same `mu.sim` and `sigma.sim` parameters that were used to generate samples from multivariate normal distributions.

```
library(MVT)
# Generate the t-samples
multt.sample <- rmvt(n = 200, delta = mu.sim, sigma = sigma.sim,
  df = 5)

# Print the first 6 samples
head(multt.sample)
```

Generate samples from multivariate t-distribution (cont.)

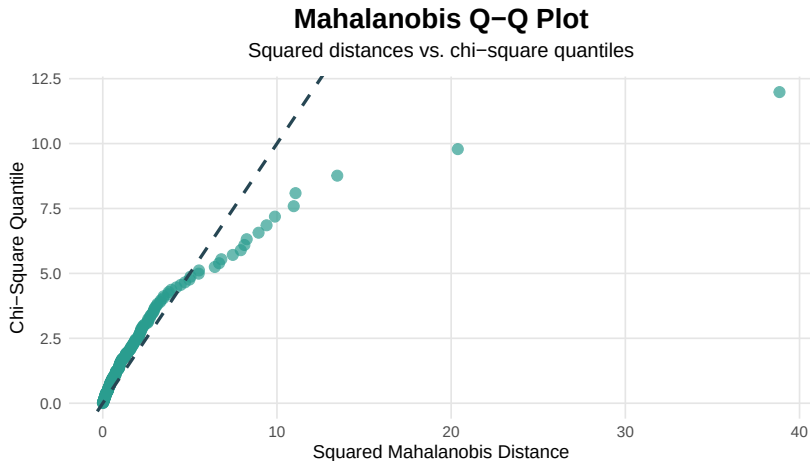
```
##           [,1]      [,2]
## [1,]  1.887816 -1.423927
## [2,]  7.486940  1.116162
## [3,]  1.620279 -1.827206
## [4,] -4.760329 -5.290355
## [5,]  9.436493  1.751836
## [6,]  2.975752 -2.299000
```

```
# Check multivariate normality
r <- mvn(multt.sample, mvn_test = "mardia")
```

```
r$multivariate_normality
```

##		Test Statistic	p.value	Method	MVN
## 1	Mardia Skewness	78.569	<0.001	asymptotic	Not normal
## 2	Mardia Kurtosis	17.196	<0.001	asymptotic	Not normal

```
plot(r, type = "qq")
```



Section 11

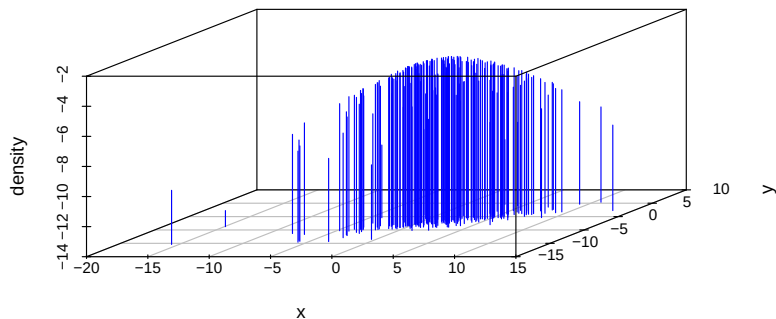
Density and cumulative density for multivariate-t

Density of multivariate t-distribution

```
# Calculate densities
multt.dens <- dmvtn(as.matrix(multt.sample), delta = mu.sim,
  sigma = sigma.sim, df = 5)

# Plot 3D heights of densities
scatterplot3d(cbind(multt.sample, multt.dens), color = "blue",
  pch = "", type = "h", xlab = "x", ylab = "y", zlab = "density")
```

Density of multivariate t-distribution (cont.)



Cumulative distributions and quantiles of t

```
# Calculate the volume under the specified  
# t-distribution  
pmvt(lower = c(-5, -5), upper = c(5, 5), delta = mu.sim,  
      df = 5, sigma = sigma.sim)
```

```
## [1] 0.6627013  
## attr("error")  
## [1] 0.0007475465  
## attr("msg")  
## [1] "Normal Completion"
```

```
# Calculate the equal probability contour  
qmvmt(p = 0.9, tail = "both", sigma = diag(2))
```

```
## $quantile  
## [1] 8.956747  
##  
## $f.quantile  
## [1] -1.35426e-07  
##  
## attr("message")  
## [1] "Normal Completion"
```

Section 12

Multivariate skew distribution

Drawing samples from skew distributions

In this exercise, you will generate random samples from a bivariate skew-normal distribution using the `rmsn()` function from the `sn` package for specified location, scale, and skewness parameters. You will then modify the code to generate random samples from a bivariate t-distribution.

```
library(sn)
# Generate the skew-normal samples
skewnorm.sample <- rmsn(n = 100, xi = mu.sim, Omega = sigma.sim,
  alpha = c(4, -4))

# Print first six samples
head(skewnorm.sample)
```

```
##           [,1]      [,2]
## [1,] 3.6032390 -1.196251
## [2,] 0.7517774 -4.266320
## [3,] 6.2349145 -0.807339
## [4,] 0.1795683 -3.203788
## [5,] 1.5415397 -3.620747
## [6,] 2.5098892 -2.048820
```

Drawing samples from skew distributions (cont.)

```
# Generate the skew-t samples
skewt.sample <- rmst(n = 100, xi = mu.sim, Omega = sigma.sim,
  alpha = c(4, -4), nu = 5)

# Print first six samples
head(skewt.sample)
```

```
##           [,1]      [,2]
## [1,] 0.4536145 -3.39326640
## [2,] 3.4308759 -1.98309856
## [3,] 7.5907955 -0.77092944
## [4,] 3.3955716 -1.22451737
## [5,] 5.0345206  0.04120972
## [6,] 2.8551076 -2.29742752
```

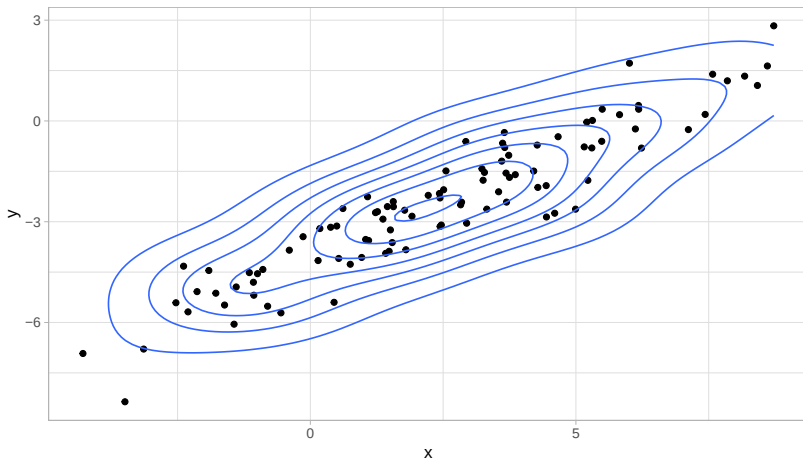
Plotting and testing of skewed-densities

To test whether a dataset follows a skew distribution, you can visualize the scatterplot and contour plot to look for non-ellipsoidal contours. You can also test whether the data follow multivariate normality.

```
skewnorm.sample <- as.data.frame(skewnorm.sample)
names(skewnorm.sample) <- c("x", "y")

# Contour plot for skew-normal sample
library(ggplot2)
ggplot(skewnorm.sample, aes(x = x, y = y)) + geom_point() +
  geom_density_2d() + theme_light()
```

Plotting and testing of skewed-densities (cont.)



```
# Normality test for skew-normal sample  
skewnorm.Test <- mvn(skewnorm.sample, mvn_test = "mardia")  
skewnorm.Test$multivariate_normality
```

Plotting and testing of skewed-densities (cont.)

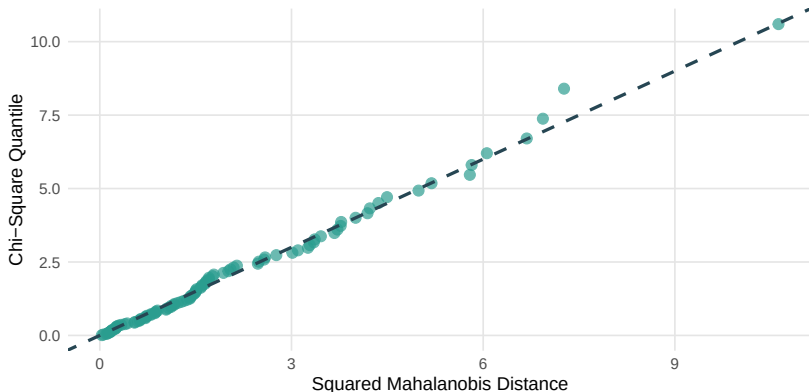
##		Test	Statistic	p.value	Method	MVN
## 1	Mardia	Skewness	2.067	0.723	asymptotic	Normal
## 2	Mardia	Kurtosis	-0.478	0.633	asymptotic	Normal

```
plot(skewnorm.Test, type = "qq")
```

Plotting and testing of skewed-densities (cont.)

Mahalanobis Q-Q Plot

Squared distances vs. chi-square quantiles



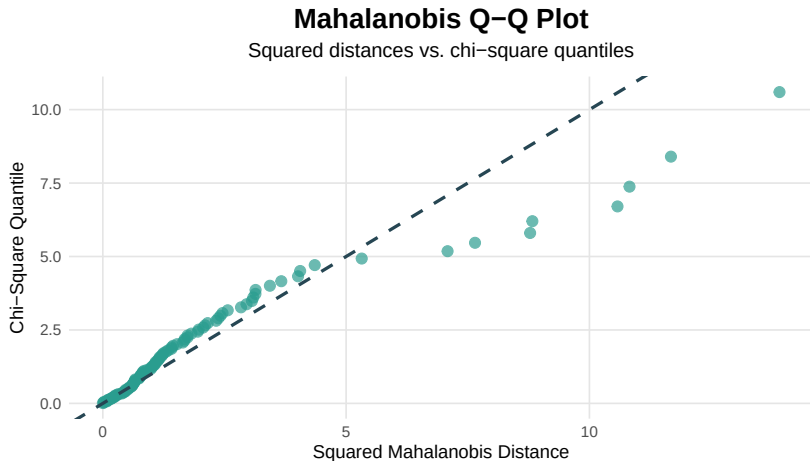
```
# Normality test for skew-t sample  
skewnorm.Test <- mvn(skewt.sample, mvn_test = "mardia")  
skewnorm.Test$multivariate_normality
```

Plotting and testing of skewed-densities (cont.)

##		Test	Statistic	p.value	Method	MVN
## 1	Mardia	Skewness	18.098	0.001	asymptotic	Not normal
## 2	Mardia	Kurtosis	3.669	<0.001	asymptotic	Not normal

```
plot(skewnorm.Test, type = "qq")
```

Plotting and testing of skewed-densities (cont.)



Parameter estimation for multivariate skew-normals

Unlike multivariate normal, where the parameters estimates can be obtained using the sample mean and sample variance-covariance matrix, the parameters of the skew-normal distribution need to be estimated by an iterative process. You can make use of existing functions in the `sn` packages to get the estimates.

```
fit.mss <- msn.mle(y = cbind(skewnorm.sample$x, skewnorm.sample$y),  
  opt.method = "BFGS")  
fit.mss
```

```
## $call  
## msn.mle(y = cbind(skewnorm.sample$x, skewnorm.sample$y), opt.method =  
  ↪ "BFGS")  
##  
## $dp  
## $dp$beta  
##           [,1]      [,2]  
## [1,] 2.094355 -1.969272  
##  
## $dp$Omega  
##           [,1]      [,2]
```

Parameter estimation for multivariate skew-normals (cont.)

```
## [1,] 8.834166 5.654933
## [2,] 5.654933 4.711663
##
## $dp$alpha
## [1] 3.616824 -3.776384
##
##
## $logL
## [1] -361.9258
##
## $aux
## $aux$alpha.star
## [1] 1.843584
##
## $aux$delta.star
## [1] 0.8790139
##
##
## $opt.method
## $opt.method$par
## [1] 2.094355 -1.969272 1.216871 -1.739759
##
## $opt.method$value
```

Parameter estimation for multivariate skew-normals (cont.)

```
## [1] 723.8516
##
## $opt.method$counts
## function gradient
##      41      14
##
## $opt.method$convergence
## [1] 0
##
## $opt.method$message
## NULL
##
## $opt.method$method
## [1] "BFGS"
##
## $opt.method$called.by
## [1] "msn.mle"
```

Section 13

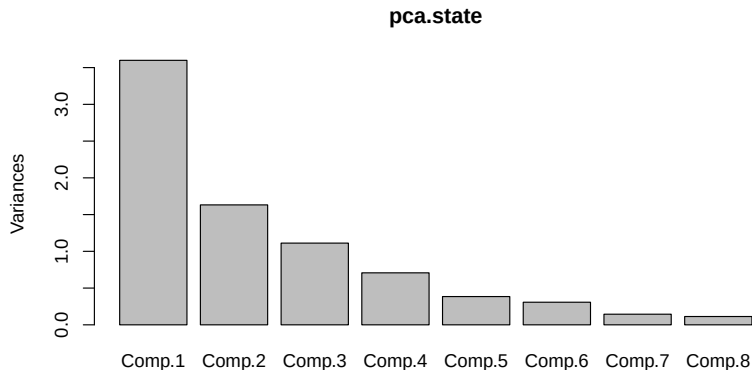
Principal Component Analysis

Using the princomp function

In this exercise you will use the `princomp()` function to calculate the principal components (PCs) of a dataset. We will use the `pca.state` object created in this exercise throughout the rest of this chapter. Calculate the principal components, and the scores of the PCs, of the `state.x77` dataset based on the correlation matrix and store it as `pca.state`.

```
# Calculate PCs  
pca.state <- princomp(state.x77, cor = TRUE, scores = TRUE)  
  
# Plot the PCA object  
plot(pca.state)
```

Using the princomp function (cont.)



```
# Print the summary of the PCs  
summary(pca.state)
```

Using the princomp function (cont.)

```
## Importance of components:
##               Comp.1    Comp.2    Comp.3    Comp.4
↪   Comp.5
## Standard deviation    1.8970755 1.2774659 1.0544862 0.84113269
↪   0.62019488
## Proportion of Variance 0.4498619 0.2039899 0.1389926 0.08843803
↪   0.04808021
## Cumulative Proportion 0.4498619 0.6538519 0.7928445 0.88128252
↪   0.92936273
##               Comp.6    Comp.7    Comp.8
## Standard deviation    0.55449226 0.3800642 0.33643379
## Proportion of Variance 0.03843271 0.0180561 0.01414846
## Cumulative Proportion 0.96779544 0.9858515 1.00000000
```

Using my own function in R

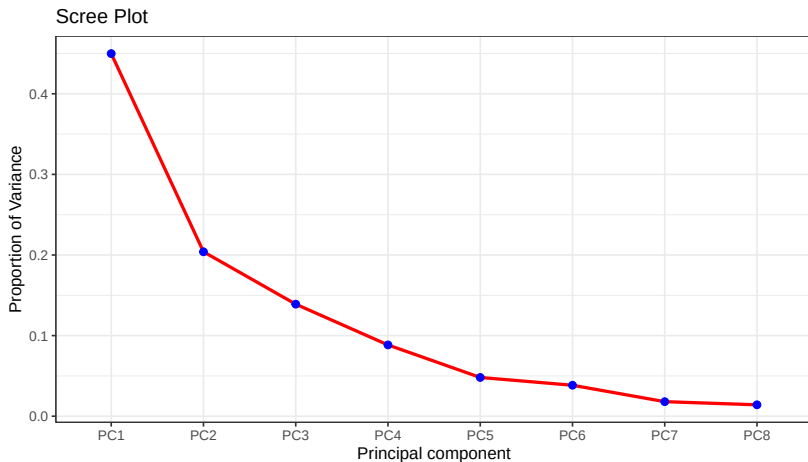
```
scaled_data <- scale(state.x77)
pc <- PCA(scaled_data)

pc$important_components
```

##	Standard Deviation	Proportion of Variance	Cumulative Proportion
## PC1	1.8970755	0.44986195	0.4498619
## PC2	1.2774659	0.20398990	0.6538519
## PC3	1.0544862	0.13899264	0.7928445
## PC4	0.8411327	0.08843803	0.8812825
## PC5	0.6201949	0.04808021	0.9293627
## PC6	0.5544923	0.03843271	0.9677954
## PC7	0.3800642	0.01805610	0.9858515
## PC8	0.3364338	0.01414846	1.0000000

```
pc$scree_plot
```

Using my own function in R (cont.)



My own function's result perfectly match with the `princomp()` function's result.

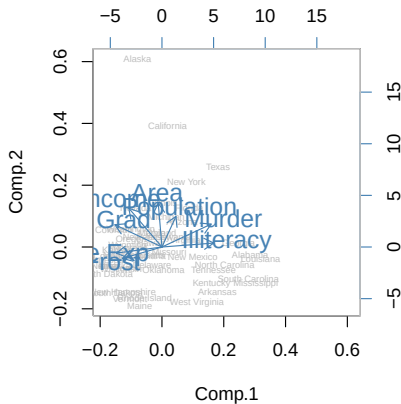
Section 14

Interpreting PCA attributes

The primary goal of PCA is dimension reduction, which is often necessary to view high dimensional data. Plotting the PC scores in two dimensions is one way to visualize high dimensional data.

```
biplot(pca.state, col = c("gray", "steelblue"), cex = c(0.5,  
1.3))
```

Loadings and scores for the PCs (cont.)



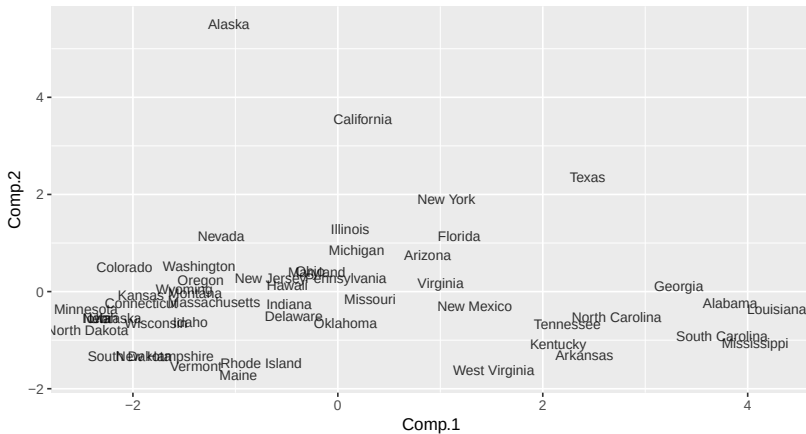
Loadings and scores for the PCs (cont.)

```
# Create data frame of scores
scores.state <- data.frame(pca.state$scores)

# Plot of scores labeled by state name
ggplot(data = scores.state, aes(x = Comp.1, y = Comp.2,
  label = rownames(scores.state))) + geom_text(alpha = 0.8,
  size = 3) + ggtitle("PCA of states data")
```

Loadings and scores for the PCs (cont.)

PCA of states data

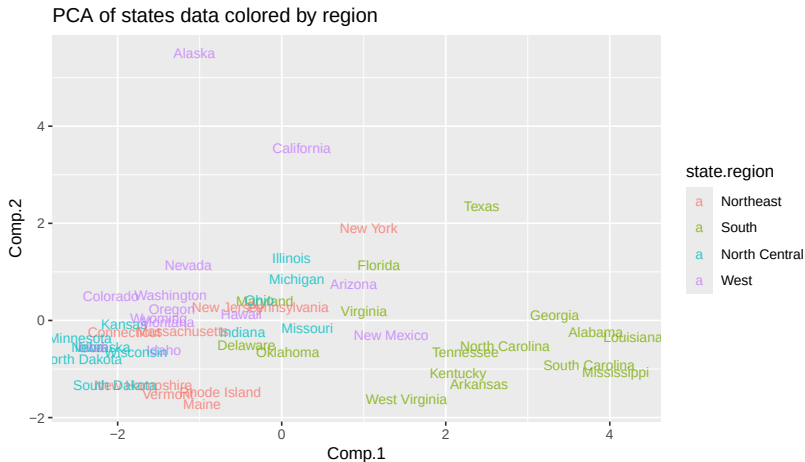


Loadings and scores for the PCs (cont.)

```
# Create data frame of scores
scores.state <- data.frame(pca.state$scores)

# Plot of scores colored by region
ggplot(data = scores.state, aes(x = Comp.1, y = Comp.2,
  label = rownames(scores.state), color = state.region)) +
  geom_text(alpha = 0.8, size = 3) + ggtitle("PCA of states data colored
  ↪ by region")
```

Loadings and scores for the PCs (cont.)



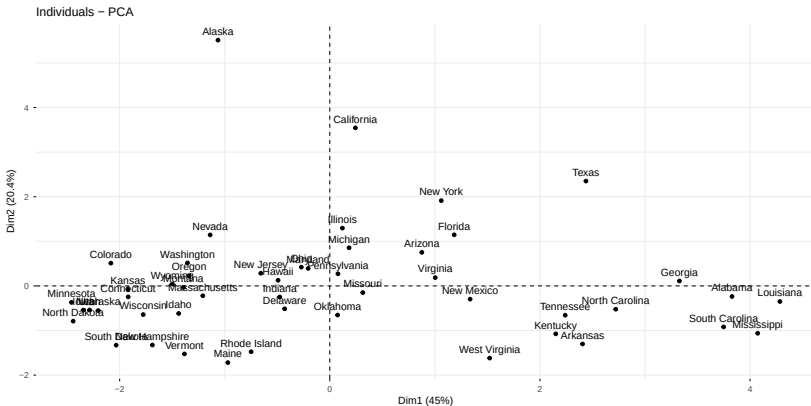
Visualizing PCA using the factoextra library

The factoextra library provides a number of functions which make it easy to extract and visualize the output of many exploratory multivariate data analyses, including PCA. In this exercise, you will use several functions to visualize the scores and PC loadings of the `state.x77` dataset.

```
library(factoextra)

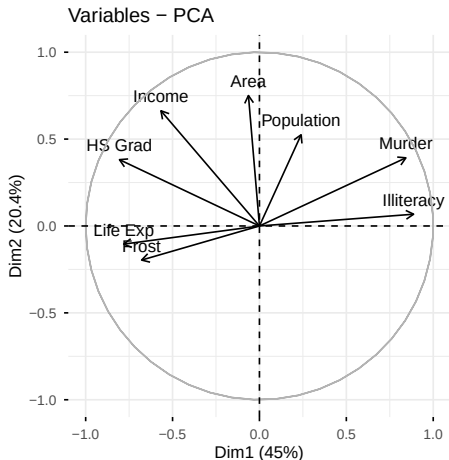
# Plot the scores
fviz_pca_ind(pca.state)
```

Visualizing PCA using the factoextra library (cont.)



```
# Plot the PC loadings
fviz_pca_var(pca.state)
```

Visualizing PCA using the factoextra library (cont.)



```
pca.state$loadings
```

Visualizing PCA using the factoextra library (cont.)

```
##
## Loadings:
##      Comp.1 Comp.2 Comp.3 Comp.4 Comp.5 Comp.6 Comp.7 Comp.8
## Population  0.126  0.411  0.656  0.409  0.406                0.219
## Income      -0.299  0.519  0.100          -0.638 -0.462
## Illiteracy   0.468                -0.353          -0.387  0.620  0.339
## Life Exp     -0.412          0.360 -0.443  0.327 -0.219  0.256 -0.527
## Murder       0.444  0.307 -0.108  0.166 -0.128  0.325  0.295 -0.678
## HS Grad      -0.425  0.299          -0.232          0.645  0.393  0.307
## Frost        -0.357 -0.154 -0.387  0.619  0.217 -0.213  0.472
## Area                0.588 -0.510 -0.201  0.499 -0.148 -0.286
##
##      Comp.1 Comp.2 Comp.3 Comp.4 Comp.5 Comp.6 Comp.7 Comp.8
## SS loadings  1.000  1.000  1.000  1.000  1.000  1.000  1.000  1.000
## Proportion Var 0.125  0.125  0.125  0.125  0.125  0.125  0.125  0.125
## Cumulative Var 0.125  0.250  0.375  0.500  0.625  0.750  0.875  1.000
```

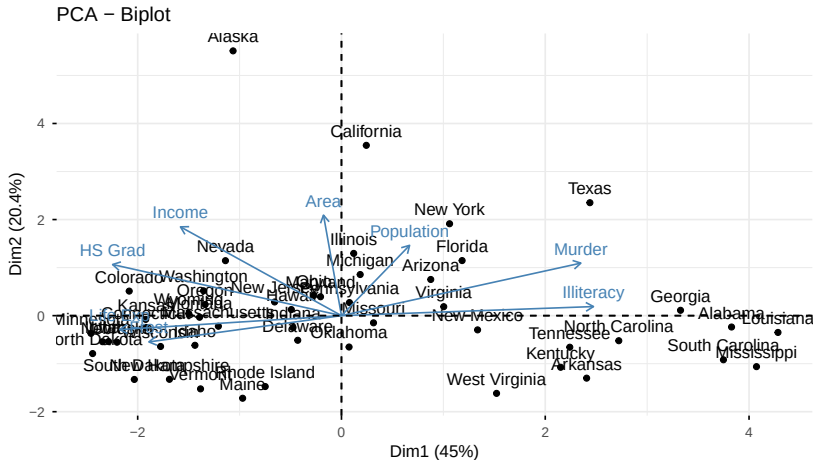
Explanation

Visualizing PCA using the factoextra library (cont.)

The PCA variable plot shows the relationships between the original variables and the first two principal components. The first principal component (Dim1) explains 45% of the total variation, while the second principal component (Dim2) explains 20.4%. Together, the first two components account for 65.4% of the total variation. Dim1 contrasts Murder, Illiteracy, and Population, which have positive loadings, with Income, HS Graduation, Life Expectancy, and Frost, which have negative loadings. Variables with arrows pointing in similar directions are positively correlated, whereas variables pointing in opposite directions are negatively correlated. For example, Income and HS Graduation are positively correlated, as are Murder and Illiteracy. In contrast, Income and Murder show a negative relationship. Dim2 is particularly associated with Area, while Life Expectancy and Frost have negative loadings on Dim2. Overall, the plot indicates that the first two principal components provide a useful summary of the relationships among the variables, explaining 65.4% of the total variation.

Visualizing PCA using the factoextra library (cont.)

```
# Create a biplot  
fviz_pca_biplot(pca.state)
```



Explanation

The PCA biplot shows the relationship between the states and the original variables using the first two principal components. Dim1 and Dim2 explain 45% and 20.4% of the total variation, respectively, accounting for 65.4% of the variation together. Dim1 mainly separates states associated with higher Murder and Illiteracy scores from those associated with higher Income and HS Graduation. Dim2 is more strongly related to variables such as Area and Population and provides additional separation among the states. Variables whose arrows point in similar directions are positively correlated, whereas variables pointing in opposite directions are negatively correlated. For example, Murder and Illiteracy show a positive association, while Income and HS Graduation are positively associated with each other and tend to contrast with Murder and Illiteracy. States located near the direction of a variable tend to have relatively high values for that variable.

Section 15

Multi-dimensional Scalling

Multidimensional scaling in two dimensions

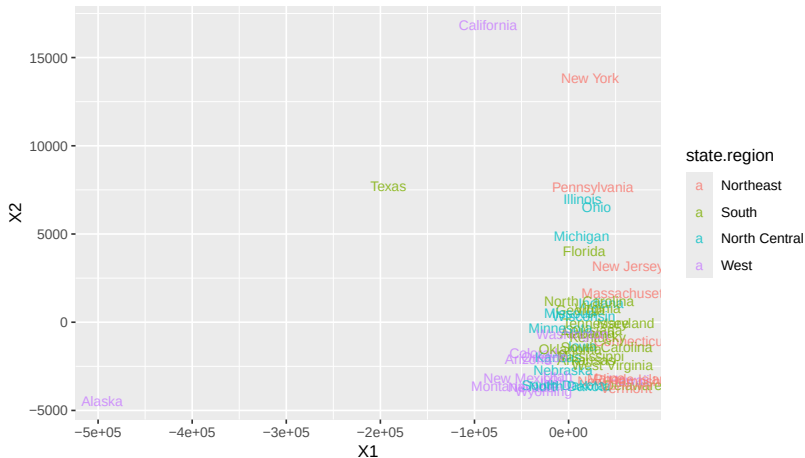
```
# Calculate distance
state.dist <- dist(state.x77)

# Perform multidimensional scaling
mds.state <- cmdscale(state.dist)

# Change mds.state to a data frame for use in ggplot
mds.state_df <- data.frame(mds.state)

# Plot the representation of the data in two
# dimensions
ggplot(data = mds.state_df, aes(x = X1, y = X2, label = rownames(mds.state),
  color = state.region)) + geom_text(alpha = 0.8, size = 3)
```

Multidimensional scaling in two dimensions (cont.)



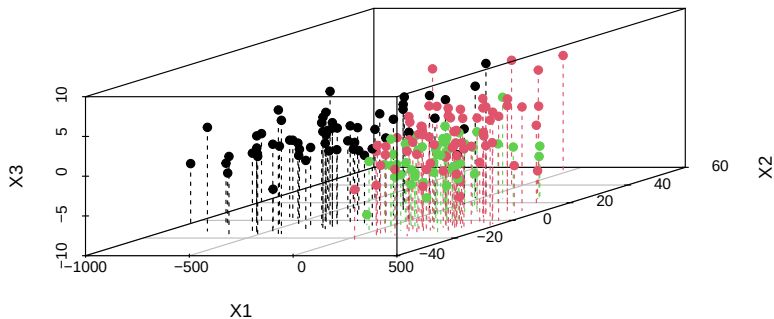
Multidimensional scaling in three dimensions

```
# Calculate distance
wine.dist <- dist(wine[, -1])

# Perform multidimensional scaling
mds.wine <- cmdscale(wine.dist, k = 3)
mds.wine_df <- data.frame(mds.wine)

# Plot the representation of the data in three
# dimensions
scatterplot3d(mds.wine_df, color = wine$Type, pch = 19,
  type = "h", lty.hplot = 2)
```

Multidimensional scaling in three dimensions (cont.)



Thank You